UNIVERSIDAD PRIVADA DEL VALLE

FACULTAD DE INFORMATICA Y ELECTRONICA

CARRERA DE INGENIERIA EN SISTEMAS INFORMÁTICOS



Actualización del Test Plan en un entorno ágil – PedidoHub

Estudiante:

- Luis Andrés Suárez Andrade

Asignatura: Aseguramiento de la Calidad del Software (SQA)

Semestre: Sexto

Grupo: A

Santa Cruz – Bolivia

Producto final	Propuesta de estrategia de automatización para un sistema web (PedidoHub)
Lectura base	Crispin, L. & Gregory, J. (2014). *Agile Testing: A Practical Guide for Testers and Agile Teams*. Parte IV: capítulos 13 y 14
Sistema analizado	PedidoHub: pedidos, reservas, facturación, reportes, inicio de sesión con roles y bitácora (ASP.NET Core MVC, EF Core, SQL Server)
Autor	Luis Andrés Suárez Andrade (la dinámica es grupal; este trabajo se presenta de forma individual)
Fecha	30 de octubre de 2026
Repositorio	https://github.com/LuisSuarez11/PedidoHub_SQA

El caso de la dinámica describe un sistema de ventas con inicio de sesión, registro de productos, ventas, descuentos y reportes, que crece cada dos semanas y se prueba a mano antes de cada entrega. PedidoHub tiene la misma forma: inicio de sesión, ventas (pedidos y facturas), cálculo de totales, reportes y entregas incrementales. Por eso la estrategia se resuelve sobre el sistema real y con datos medidos, no con ejemplos inventados.

Fase 1. Detectives de la automatización

Beneficios (con respaldo del libro y evidencia en PedidoHub)

#	Beneficio	Idea del libro (cap. 13)	Evidencia en PedidoHub
B1	**Retroalimentación rápida y defectos detectados temprano.** El desarrollador se entera del error minutos después de escribirlo, cuando corregirlo es barato.	La automatización da respuesta rápida sobre si el código sigue funcionando y evita que los errores se acumulen hasta el final de la iteración.	180 pruebas unitarias y de integración corren en unos 13 segundos. Cinco de los seis defectos (reserva en el pasado, dos pedidos en una mesa, factura con fecha anterior, falta de `/health` y de cabeceras de seguridad) se descubrieron escribiendo la prueba antes del código.
B2	**Libera tiempo para lo que solo una persona puede hacer.** La regresión repetitiva deja de ser trabajo manual y el tiempo se usa en pruebas exploratorias.	Automatizar las pruebas repetitivas permite que las personas dediquen su tiempo a explorar, pensar casos nuevos y evaluar la experiencia de uso.	La regresión de 17 historias se ejecuta con 216 pruebas automáticas; las sesiones manuales quedan para el plano de mesas y la usabilidad.
B3	**Red de seguridad para cambiar el sistema y documentación ejecutable.** Se puede refactorizar o agregar funciones sabiendo de inmediato si algo se rompió, y los nombres de las pruebas describen el comportamiento esperado.	Las pruebas automatizadas dan confianza para modificar el código y sirven como documentación viva del comportamiento.	Al agregar inicio de sesión, roles y bitácora, la suite avisó en segundos qué pantallas se afectaron. Las pruebas se llaman `Dado_…_Cuando_…_Entonces_…`, de modo que se leen como requisitos.

Obstáculos (con respaldo del libro y evidencia en PedidoHub)

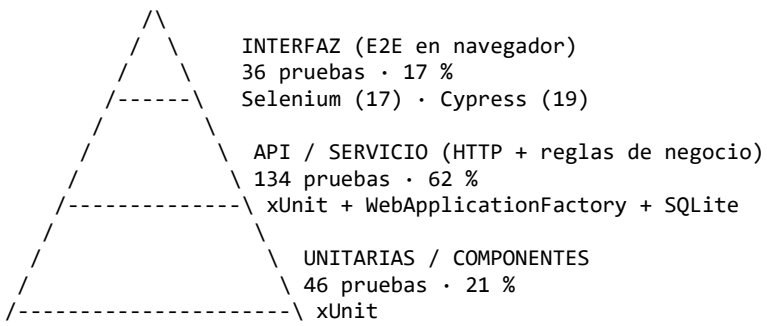
#	Obstáculo	Idea del libro (caps. 13 y 14)	Evidencia en PedidoHub
O1	**Inversión inicial y curva de aprendizaje.** Al principio automatizar cuesta más tiempo que probar a mano y parece que no se avanza.	El libro describe una "joroba" de esfuerzo inicial y la dificultad de dedicar tiempo a aprender y preparar herramientas y entornos.	Preparar la infraestructura tomó varias iteraciones: SQLite no suma decimales y hubo que adaptar el modelo de pruebas; Cypress falló al principio por un archivo con BOM. Esa inversión se hizo una vez y ahora cada prueba nueva es barata.
O2	**Pruebas frágiles y costo de mantenimiento, sobre todo en la interfaz gráfica.** Un cambio visual rompe pruebas que no tenían un defecto real.	Las pruebas de interfaz son lentas y frágiles; si se concentra ahí la automatización, el mantenimiento consume el beneficio.	Al agregar la landing pública y el login, las pruebas E2E dejaron de funcionar hasta cambiar la ruta de inicio y agregar el inicio de sesión en un único método auxiliar. Un selector mal escrito en Cypress produjo un falso rojo.
O3	**Entornos y datos de prueba distintos de producción, y la idea de que automatizar es "cosa de testers".** Si las pruebas corren en condiciones diferentes, dan una falsa seguridad.	Hace falta apoyo de todo el equipo y de un entorno que permita probar de forma repetible; las pruebas deben pensarse como parte del producto.	Las pruebas de integración usan SQLite en memoria mientras producción usa SQL Server. Se mitigó ejecutando los E2E contra SQL Server real en el CI y manteniendo datos iniciales conocidos.

Respuestas a las preguntas de la fase

- **¿Por qué se automatizan las pruebas?** Para obtener retroalimentación rápida y repetible, evitar que la regresión manual crezca con cada entrega y poder cambiar el sistema con confianza.
- **¿Qué tareas manuales consumen demasiado tiempo?** Repetir en cada entrega los flujos ya probados: crear un pedido, facturarlo, verificar totales, validar permisos por rol y comprobar reportes.
- **¿Cómo ayuda a detectar defectos temprano?** Una prueba escrita antes del código falla (rojo) hasta que la regla existe; cualquier regresión posterior hace fallar el CI en el mismo día.
- **¿Qué dificultades impiden automatizar?** Falta de tiempo y de habilidades, pruebas frágiles, entornos distintos de producción y la creencia de que la automatización reemplaza todo lo manual.

Fase 2. Construcción de la pirámide de pruebas

Pirámide de PedidoHub (conteo real de la última ejecución)



Total: 216 pruebas, todas en verde. Cobertura de líneas de la lógica: 81.3 % (1328 de 1633).

Ejemplos por nivel y justificación de su ubicación

Nivel	Ejemplos reales de PedidoHub	Por qué van en este nivel
Unitarias y de componentes (base)	1) `Pedido_Total_SumaLosSubtotalesDeSusPlatos`: el total del pedido es Σ cantidad × precio. 2) `Factura_Nit_SoloAceptaDigitosEntre5Y20`: validación del NIT con valores límite. 3) `ReporteService`: ranking de platos con desempate por nombre.	Verifican una regla pequeña sin base de datos ni navegador. Son las más rápidas (milisegundos) y señalan exactamente qué se rompió.
API / servicio (medio)	1) `Dada_una_mesa_con_pedido_sin_facturar_Cuando_se_crea_otro…_se_rechaza`: regla de negocio a través de los controladores y la base de datos. 2) `Dado_un_mesero_Cuando_abre_Empleados_Entonces_se_le_deniega`: permisos por rol. 3) `VentasCsv` devuelve un CSV con las 4 facturas.	PedidoHub es una aplicación MVC sin API JSON: las reglas viven en los controladores. Probarlas por HTTP sin interfaz es el equivalente a la capa de servicio: ejercita reglas, seguridad y datos juntos y sigue siendo rápido (13 s para 180 pruebas).
Interfaz gráfica (cima)	1) Selenium `E2E_06`: reservar 5 personas en una mesa de 4 muestra el error. 2) Cypress: iniciar sesión, abrir el panel de calidad y comprobar que un mesero recibe "Acceso denegado".	Comprueban que las capas funcionan unidas en un navegador real (formularios, JavaScript, cookies). Son lentas y frágiles, por eso son pocas y solo cubren flujos críticos.

Preguntas orientadoras

- **¿Por qué la base suele contener más pruebas?** Porque son baratas, rápidas y estables; permiten cubrir muchos casos y valores límite. En PedidoHub, 180 de 216 pruebas (83 %) no abren un navegador.
- **¿Qué desventaja tendría automatizar casi todo por la interfaz?** Suites lentas, frágiles ante cualquier cambio visual, difíciles de diagnosticar (no se sabe qué capa falló) y caras de mantener. Por eso no se repiten en Selenium las reglas que ya cubren las 134 pruebas de la capa media.

- **¿Qué pruebas dan retroalimentación más rápida al desarrollador?** Las unitarias y de integración: las 180 corren en unos 13 segundos, antes de hacer *push*.
- **Desviación consciente del modelo:** la pirámide ideal tiene una capa media más pequeña. Aquí es la mayor porque no hay API separada y la lógica está en los controladores. Se justifica por propósito: cada regla se prueba en el nivel más bajo en que se puede probar sin perder realismo.

Fase 3. Laboratorio de decisiones SQA

Matriz de decisiones (las tres primeras pruebas a automatizar)

Prioridad	Prueba	Nivel y herramienta	Resultado esperado	Por qué primero	Estado en PedidoHub
1	Cálculo del total de un pedido y de la factura (cantidad × precio, suma de platos, ticket promedio)	Unitaria · xUnit	Pedido 1 = 130.00; un pedido sin platos = 0; promedio de 4 facturas = 150.75	Es dinero: un error afecta al cliente y a la contabilidad. Es el cálculo que más se repite en cada cambio. Es barata y estable.	`CalculosDominioTests`, `ReporteServiceTests`
2	Reglas de facturación y pedidos (no doble factura, no facturar sin platos, una cuenta abierta por mesa, fecha de emisión coherente)	Integración por HTTP · xUnit + WebApplicationFactory + SQLite	El sistema rechaza la operación con el mensaje exacto y la base de datos no cambia	Concentra los defectos: tres de los seis encontrados estaban en estas reglas (reserva en el pasado, dos pedidos por mesa, factura con fecha anterior). Cada nueva función las puede romper.	`ReglasDeNegocioTests`
3	Inicio de sesión y permisos por rol	Integración por HTTP · xUnit, más un flujo de humo en Selenium y Cypress	Sin sesión → pantalla de login; mesero en Empleados → "Acceso denegado"; clave incorrecta → mensaje genérico	Un fallo aquí expone todo el sistema. Es la puerta de entrada de cada prueba E2E posterior.	`SeguridadTests`, `E2E-13` a `E2E-17`, Cypress "Acceso, landing y roles"

Elección de herramientas

Necesidad	Herramienta	Justificación
Pruebas unitarias y de integración	xUnit con `WebApplicationFactory`	Mismo lenguaje que el sistema (C#), arranca la aplicación en memoria y es rápida
E2E con ecosistema .NET	Selenium WebDriver	Comparte código y datos con el resto de la solución; controla un navegador real

Necesidad	Herramienta	Justificación
E2E con ciclo de depuración rápido	Cypress	Reintentos automáticos, capturas al fallar y verificación HTTP directa
Cobertura	coverlet	Se integra con `dotnet test` y alimenta el Panel de calidad
Ejecución continua	GitHub Actions	Corre en cada *push* sin infraestructura propia

¿Qué se deja manual y por qué?

Prueba manual	Motivo
Sesiones exploratorias del plano de mesas con pedidos simultáneos	Requieren criterio humano y descubren lo que nadie pensó en probar
Usabilidad y diseño visual (legibilidad, jerarquía, vista móvil)	Una máquina no juzga si algo es claro o agradable
Pruebas de aceptación con el usuario real (flujo del mesero en un turno)	El valor está en la percepción de quien usa el sistema
Revisión de textos y mensajes	Es un juicio de redacción y de tono
Pruebas de carga y de recuperación ante fallos	No son rentables de automatizar en esta etapa; se registran como mejora futura

¿Cómo evitar que las pruebas se vuelvan difíciles de mantener?

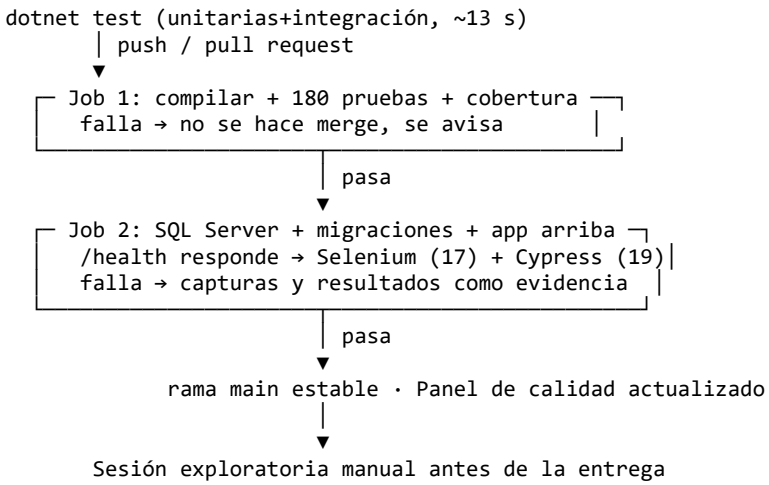
- **Una sola prueba por regla y en el nivel más bajo posible**, para no duplicar esfuerzo en la interfaz.
- **Código de apoyo reutilizable** (`PruebaIntegracion`, `PruebaNavegador`, `ingresar()` en Cypress): cuando cambió el inicio de sesión se corrigió en un solo lugar.
- **Pruebas independientes**: cada una crea su propia base en memoria, o usa nombres únicos; no dependen del orden.
- **Selectores estables** (`name` de los campos) y esperas explícitas, nunca pausas fijas.
- **Nombres que describen el comportamiento** (`Dado/Cuando/Entonces`) para que un fallo se entienda sin leer el código.
- **Una prueba roja se arregla el mismo día**; nunca se deshabilita sin dejar una tarjeta en el tablero.
- **Revisión en cada retrospectiva**: se eliminan las pruebas obsoletas y se revisan las métricas (tasa de éxito, cobertura, tiempo de la suite).

Fase 4. Integración continua: ¿cuándo ejecutar las pruebas?

Flujo

```
Desarrollador          Repositorio (GitHub)                    Equipo
─────────────          ────────────────────                    ──────

escribe prueba (rojo)
implementa (verde)
```



Qué se ejecuta y cuándo

Momento	Pruebas	Tiempo (estimado)	Objetivo
Antes de hacer *push* (en la máquina)	Unitarias e integración	13 s	Feedback inmediato al desarrollador
Cada *push* y *pull request*	Compilación, unitarias, integración y cobertura (Job 1)	1–2 min	Impedir que entre código roto
Tras pasar el Job 1	E2E con Selenium y Cypress contra SQL Server (Job 2)	3–5 min	Validar los flujos críticos con la base real
Antes de cada entrega	Sesión exploratoria y revisión de usabilidad	1 h	Lo que no se automatiza
Cierre de sprint	Auditoría, métricas y retrospectiva	—	Mejora continua

¿Qué hacer cuando una prueba falla?

- **Detenerse:** un *pipeline* en rojo significa que no se hace *merge*; no se sigue agregando código sobre un fallo (la protección de rama en GitHub queda como mejora).
- **Clasificar la causa** en menos de 15 minutos: (a) defecto real del sistema, (b) prueba mal escrita o frágil, (c) problema de entorno o de datos.
- **Corregir el mismo día**, por quien hizo el cambio: si es (a) se arregla el código; si es (b) se arregla la prueba; si es (c) se corrige el entorno.
- **No ignorar ni desactivar** una prueba para "que pase". Si falla de forma intermitente se reproduce, se estabiliza y se anota en el tablero.
- **Agregar una prueba de regresión** para que el mismo error no vuelva.
- **Aprender en la retrospectiva:** ¿por qué no lo detectó un nivel más bajo de la pirámide?

Ejemplo real: tras agregar el inicio de sesión, el CI de Cypress falló 8 pruebas. Se clasificó como (b): faltaba definir la función de inicio de sesión en el archivo de pruebas. Se corrigió, pasaron las 19 y no hubo que tocar el sistema.

Fase 5. Defensa del equipo QA

Guion de 15 minutos

Minuto	Contenido
0–2	Problema: el caso de ventas que crece cada dos semanas y la regresión manual que cada vez tarda más. Presentar PedidoHub como el sistema real.
2–5	Fase 1: tres beneficios y tres obstáculos con la evidencia propia.
5–8	Fase 2: la pirámide con los números reales (46 / 134 / 36) y por qué la capa media es grande.
8–11	Fase 3: matriz de decisiones: las tres pruebas, la herramienta, el resultado esperado y qué queda manual.
11–13	Fase 4: flujo de CI y qué se hace cuando una prueba falla. Mostrar el Panel de calidad en `/Calidad`.
13–15	Conclusiones y preguntas.

Preguntas de defensa y respuestas

1. ¿Por qué eligieron esas pruebas y no otras? Se priorizó por riesgo, frecuencia de cambio y costo. El cálculo de totales y las reglas de facturación manejan dinero y se tocan en casi cada entrega. El inicio de sesión y los roles protegen todo el sistema. Los datos lo confirman: cuatro de los seis defectos hallados (tres de reglas de negocio y uno de seguridad) estaban en lo que cubren las pruebas 2 y 3. Lo que no se eligió (diseño visual, carga) o no era rentable en esta etapa o necesita criterio humano.

2. ¿Qué beneficio concreto esperan obtener? Retroalimentación en 13 segundos en lugar de esperar a una regresión manual. Los E2E completos tardan alrededor de un minuto. Una regresión manual de las 17 historias me tomaría del orden de una hora (es una estimación, no una medición). Además, la bitácora de defectos muestra seis defectos corregidos y todos con una prueba de regresión.

3. ¿Qué riesgos existen si la prueba automatizada está mal diseñada? Falsos verdes que dan una confianza que no corresponde, falsos rojos que hacen perder tiempo y que el equipo termine ignorando el CI, pruebas que dependen unas de otras o de datos compartidos, y mantenimiento que cuesta más que el beneficio. Lo viví: SQLite se comporta distinto de SQL Server con los decimales (mitigado con E2E contra SQL Server) y un selector mal escrito en Cypress produjo un falso rojo.

4. ¿Qué harían si cambia frecuentemente la interfaz? Mover la verificación hacia abajo en la pirámide: las reglas se prueban por HTTP, sin depender de la pantalla. En la interfaz se dejan pocas pruebas de flujos críticos, con selectores por `name` y código de apoyo compartido. Cuando se cambió la ruta de inicio y se agregó el login, se corrigió en un solo método auxiliar en cada herramienta.

5. ¿Cómo comprobarían que el esfuerzo de automatización vale la pena? Con métricas del Panel de calidad: defectos detectados por las pruebas antes de liberar, tiempo de retroalimentación, tasa de éxito, cobertura (81.3 %), proporción de pruebas inestables y tiempo dedicado a mantener la suite. Si una prueba cuesta más mantener que el defecto que previene, se elimina o se mueve a un nivel más bajo.

Cierre de la dinámica

- **¿Qué comprendí sobre la automatización que antes no tenía claro?** Que automatizar no es escribir scripts ni elegir una herramienta: es decidir qué probar, en qué nivel y cuándo ejecutarlo. Mi primera idea era cubrir todo con pruebas de navegador; los datos del proyecto muestran que la mayor parte del valor está en los niveles bajos.
- **¿Qué prueba automatizaría primero en un proyecto real y por qué?** El cálculo de totales y las reglas de facturación, porque involucran dinero, cambian con frecuencia y son baratas y estables de probar.
- **¿Qué error cometería un equipo que cree que automatizar es eliminar todas las pruebas manuales?** Dejaría de explorar: se perderían los problemas de usabilidad y los casos que nadie previó, y las pruebas automáticas solo comprobarían lo que ya se sabía. La automatización libera tiempo para pensar mejor, no sustituye al criterio de una persona.

Cómo se evidencia cada criterio de evaluación

Criterio (20 puntos cada uno)	Dónde se evidencia en este trabajo
Comprensión del documento	Fase 1: tres beneficios y tres obstáculos con ideas de los capítulos 13 y 14
Aplicación de la pirámide	Fase 2: pirámide con conteo real y ejemplos por nivel con su justificación
Selección de pruebas y herramientas	Fase 3: matriz de decisiones, elección de herramientas y qué queda manual
Estrategia de automatización	Fases 3 y 4: flujo de CI, retroalimentación, qué hacer ante un fallo y mantenimiento
Exposición y defensa	Fase 5: guion de 15 minutos y respuestas a las cinco preguntas

Evidencia reproducible

Todo lo citado se puede verificar en el repositorio https://github.com/LuisSuarez11/PedidoHub_SQA:

- `PedidoHub.Tests/` (unitarias e integración), `PedidoHub.E2E/` (Selenium) y `cypress/`.
- `.github/workflows/ci.yml` (flujo de integración continua).
- `Docs/evidencias/` (resultados y cobertura) y `PedidoHub/Data/calidad.json` (datos del Panel de calidad en `/Calidad`).

Bibliografía

- Crispin, L. & Gregory, J. (2014). *Agile Testing: A Practical Guide for Testers and Agile Teams*. Addison-Wesley. Parte IV, capítulos 13 y 14.
- Cohn, M. (2009). *Succeeding with Agile*. Addison-Wesley (origen de la pirámide de automatización).
- ISTQB Foundation Level Syllabus (v4.0, 2023).
- Fowler, M. (2019). *Continuous Integration and Delivery*. ThoughtWorks.